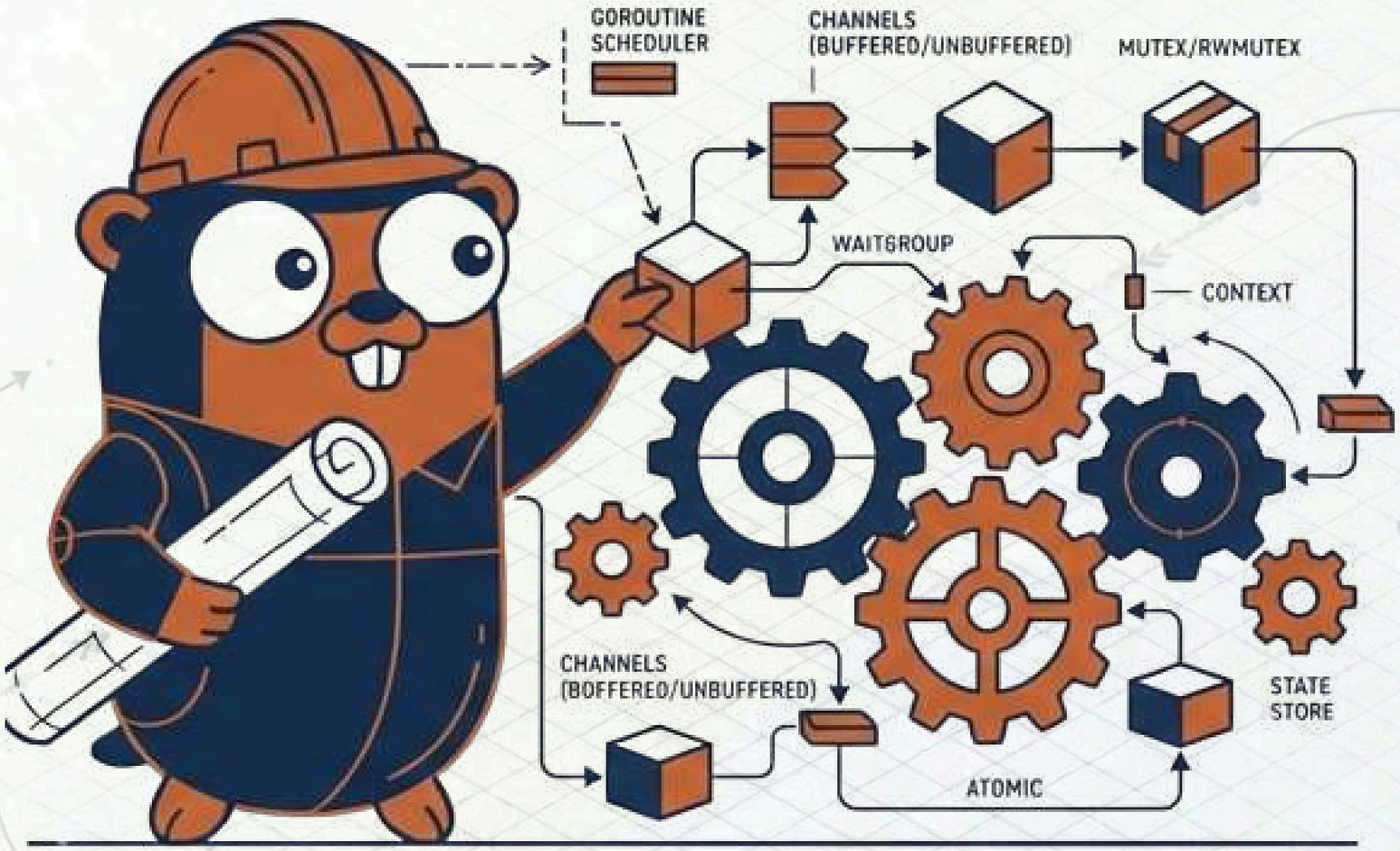
CONCURRENCY IN GO



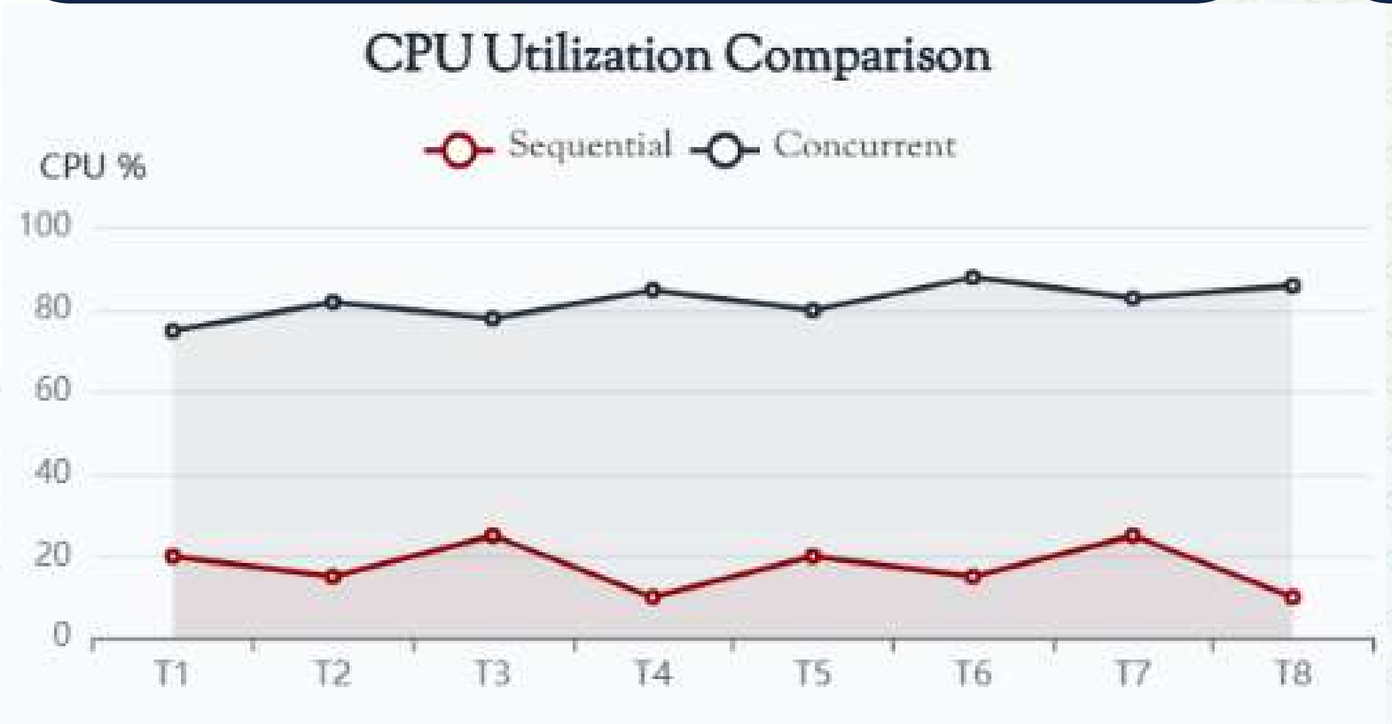
WHY CONCURRENCY?

1.Sequential Bottlenecks:

Traditional execution blocks the CPU while waiting for I/O operations (network,disc,database).

2.System Underutilization:

Modern processors have multiple cores. Single-threaded applications leave applications idle.



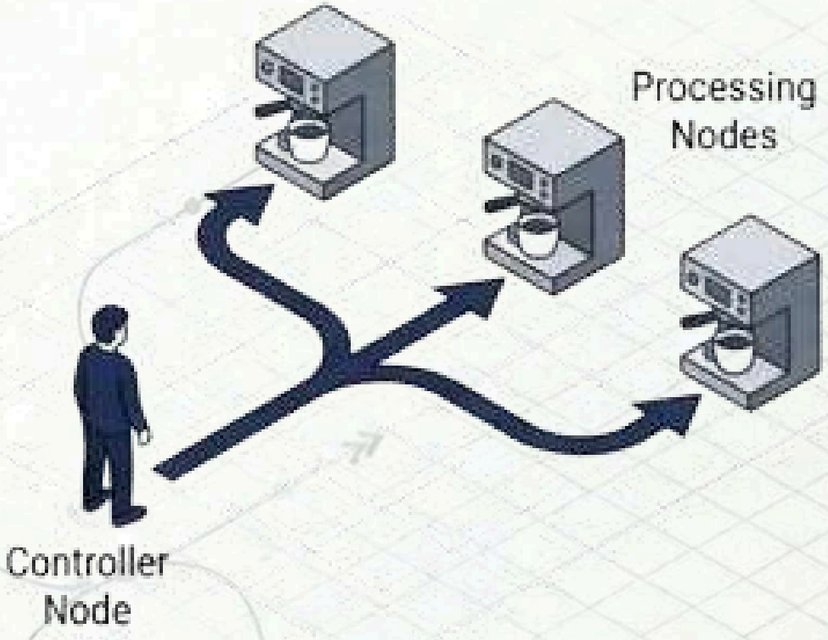
CONCURRENCY

Concurrency is about optimizing throughput by keeping the CPU continuously busy. When one task blocks on I/O, the system switches to another ready task, maximizing resource utilization.

CONCURRENCY VS PARALLELISM

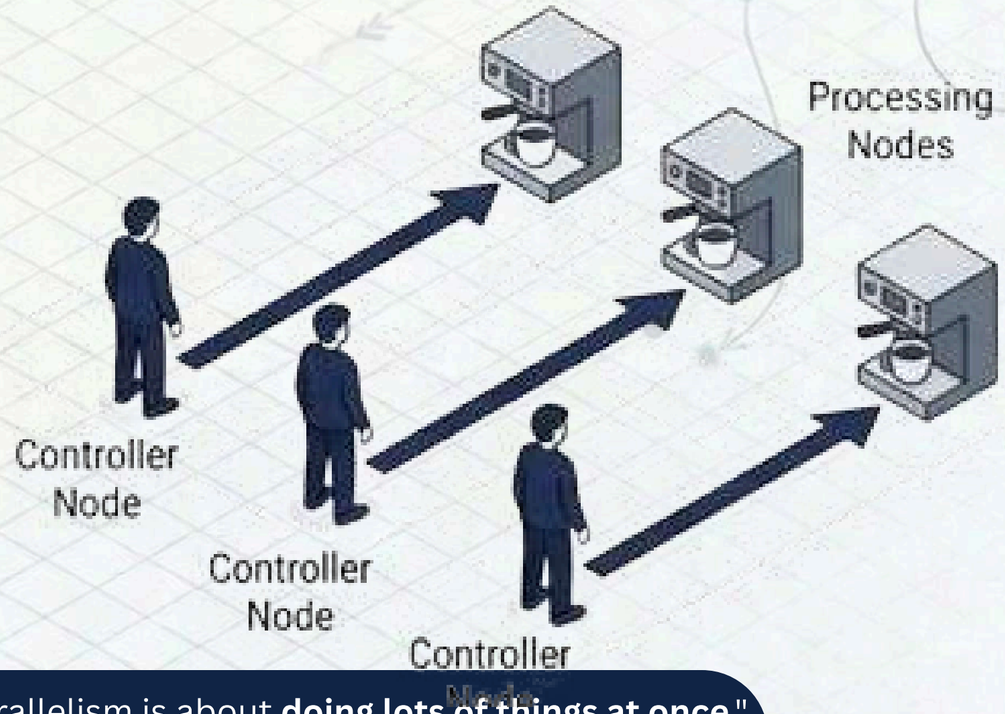
Concurrency

The composition of independently executing processes (managing the workflow).



Parallelism

The simultaneous execution of multiple computations (hardware executing it).

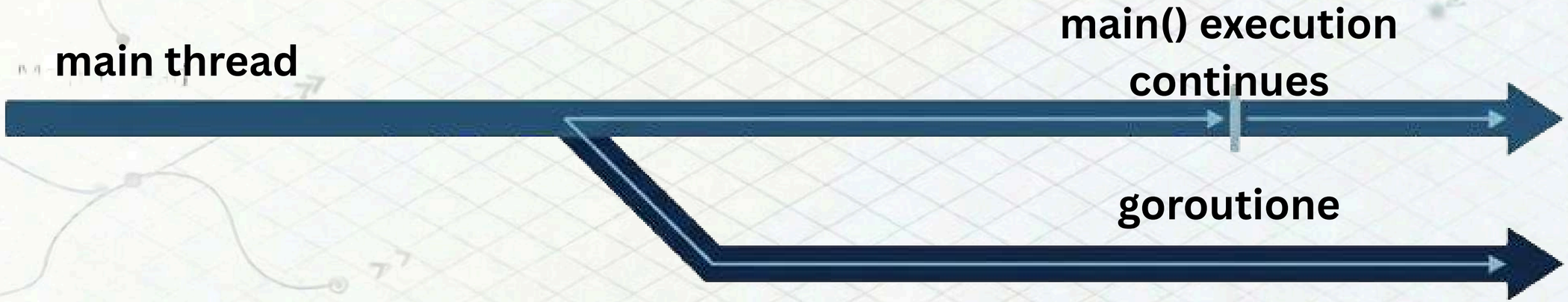


"Concurrency is about **dealing with lots of things at once**. Parallelism is about **doing lots of things at once**."
— Rob Pike, Co-creator of Go

Goroutines

- Managed by the Go runtime, not the OS.
- Extremely lightweight (~2KB initial stack).
- Run hundreds of thousands concurrently without crashing the system.

```go
//sequential call
PerformTask()
//Launch a Goroutine
go PerformTask()
//Fire and Forget
go func(){
  fmt.Println("Running Concurrently")
}()
```



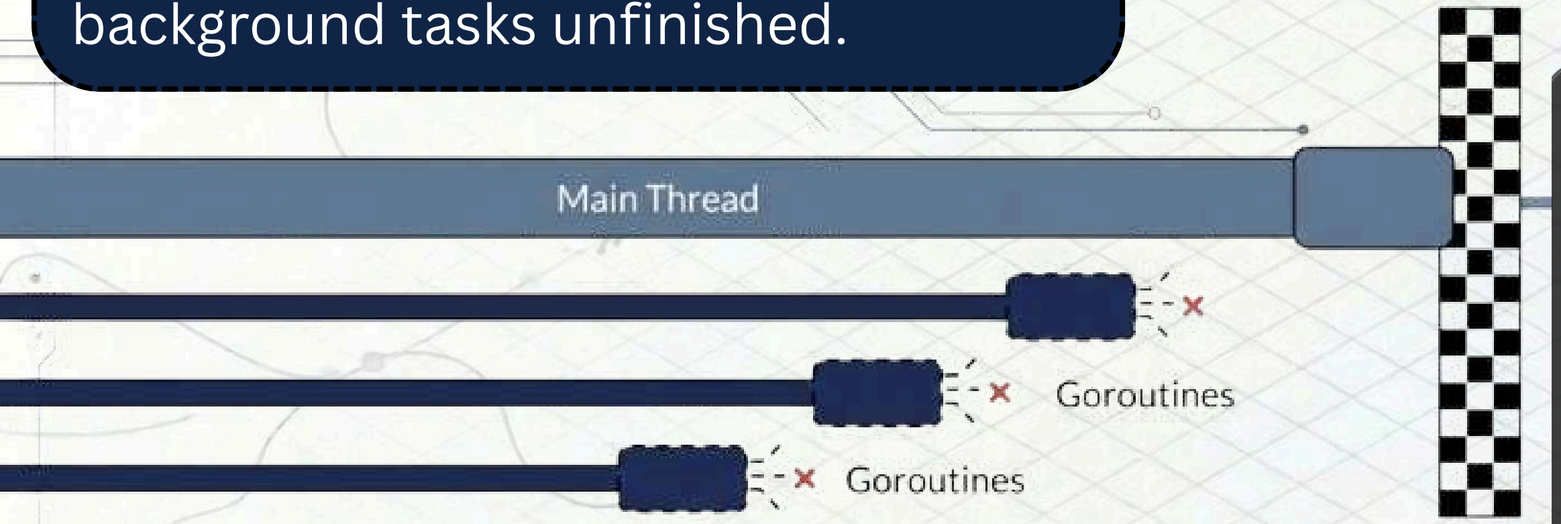
WaitGroups

Problem with Goroutine

When the main thread generates goroutines, it exits immediately without waiting for them to complete. This causes the program to terminate prematurely, potentially leaving background tasks unfinished.

Waitgroups

A thread-safe counter to block the main thread until workers finish.



waitgroups

```go
var wg sync.WaitGroup //intialization
wg.Add(1) //counter of waitgroups
go func(){
  defer wg.Done()//decrement when finished
  dowork()
}
wg.Wait() //blocks mainthread till counter zero
```

CHANNELS AND MUTEXES

Channels

Channels are typed, thread-safe conduits that connect concurrent goroutines. They enable safe communication by passing ownership of data, eliminating race conditions without explicit locks

```go
// Create a channel of strings
messages := make(chan string)

//  Goroutine sending data to the channel
go func() {
    messages <- "Hello,World"
}()

// 2. Main thread receiving data from the channel
msg := <-messages
```

Mutexes

A mutex is a synchronization mechanism that restricts access to a shared resource to one thread at a time, preventing race conditions and ensuring data integrity.

```go
var mu sync.Mutex
var counter int

func update() {
    mu.Lock()   // Locks the code
    counter++   // code to be locked
    mu.Unlock() // Releases the lock
}
```

"Do not communicate by sharing memory; instead, share memory by communicating.."
— Rob Pike, Co-creator of Go

CONCURRENCY IN GO

```go
var totalcalls int

func fetchprice(Airline string){
    totalcalls++
    delay:=time.Duration(rand.Intn(300))*time.Millisecond// random delay in time
    time.Sleep(delay)
    time.Sleep(delay)
    price :=(rand.Intn(5000)+8000)
    fmt.Printf("%s->%d time:%v\n",Airline,price,delay)
}
func main() {
   Airlines:=[]string{"Indigo","Air India","Vistara","SpiceJet","Akasa Air"}
   start:=time.Now()
   for _, Airline:= range Airlines{
    fetchprice(Airline)// normal function call, runs one by one
    }
   }
   fmt.Printf("Time Taken:%v\n",time.Since(start))
   fmt.Printf("Calls Taken:%d\n",totalcalls)
}
```

Flight Tracker without Concurrency

CONCURRENCY IN GO

```go
var (
    totalcalls int
    mu     sync.Mutex  //Mutex declartion
)
func fetchprice(Airline string, wg *sync.WaitGroup,result chan<-string){
    defer wg.Done()//once function is executed it will decrement wait group counter
    mu.Lock() // locks totalcalls variable
    totalcalls++
    mu.Unlock()//unlocks totalcalls variable
    delay:=time.Duration(rand.Intn(300))*time.Millisecond //random delay in time
    time.Sleep(delay)
    price :=(rand.Intn(5000)+8000)
    result <- fmt.Sprintf("%s->%d time:%v\n",Airline,price,delay)//transfer through channel

}
```

Price Fetcher Function with Concurrency

CONCURRENCY IN GO

```go
func main() {
   Airlines:=[]string{"Indigo","Air India","Vistara","SpiceJet","Akasa Air"}
   var wg sync.WaitGroup //waitgroup declartion
   start:=time.Now()
   result := make(chan string,len(Airlines))//channel declartion
   for _, Airline:= range Airlines{
    wg.Add(1)
    go fetchprice(Airline,&wg,result) //goroutine
   }
   wg.Wait() // wait group will till its counter becomes zero
   close(result)//closes the channel
   for  results:= range result{
    fmt.Println(results)
   }
   fmt.Printf("Time Taken:%v\n",time.Since(start))
   fmt.Printf("Calls Taken:%d\n",totalcalls)
}
```

Main Function with Concurrency

CONCURRENCY IN GO

```
Indigo->11780 time:178ms
Air India->8577 time:245ms
Vistara->12792 time:212ms
SpiceJet->12130 time:112ms
Akasa Air->9133 time:0s
Time Taken:750.5462ms
Calls Taken:5
```

```
Air India->11708 time:13ms

Indigo->11567 time:58ms

SpiceJet->10789 time:88ms

Vistara->10759 time:197ms

Akasa Air->10041 time:256ms

Time Taken:257.082ms
Calls Taken:5
```

Result of both Code

THANK YOU

By Mehta Param
25IM10013